

Table of Contents

1 Informe de tests de `xtendoo-pos-conventional`

Repositorio analizado: `odoo/custom/src/xtendoo-pos-conventional` **Fecha del análisis:** 2026-05-14 **Documento fuente canónico:** este fichero Markdown
Artefactos derivados recomendados: PDF y DOCX generados desde este fichero

1.1 1. Resumen ejecutivo

En `xtendoo-pos-conventional` hay actualmente:

- **22 archivos de test** `test_*.py`
- **528 métodos de test** inventariados en total
- **2 tests frontend HOOT** del propio repositorio

1.1.1 Resultado real verificado

1.1.1.1 Backend

Se han ejecutado **526 tests backend** directamente dentro de `odoo shell` usando `unittest` sobre las clases del repositorio.

Resultado backend verificado:

- **526 ejecutados**
- **13 fallos**
- **11 errores**
- **22 clases ejecutadas**
- **1 skipped** observado en `pos_conventional_order_barcode/tests/test_order_barcode.py`

El resto de la suite backend quedó **OK**.

1.1.1.2 Frontend / HOOT

Hay **2 tests frontend HOOT** del repositorio:

1. `pos_conventional_core/tests/test_product_label_section_and_note_field_js.py`
2. `pos_conventional_order_barcode/tests/test_order_barcode_frontend.py`

Estado verificado:

- No se ha conseguido una ejecución **limpia e aislada** de esos 2 tests en este entorno.
- Ejecutados desde `odoo shell`, fallan en `setUpClass` porque `HttpCase/HOOT` necesita un servidor HTTP vivo:

`AttributeError: 'NoneType' object has no attribute 'server_port'`

- En una ejecución previa con el runner nativo de Odoo, `TestPosOrderBarcodeFrontend` sí llegó a arrancar, pero la ejecución quedó contaminada por suites HOOT upstream de `@web/...`, así que ese resultado no puede considerarse una validación limpia del código del repositorio.
-

1.2 2. Metodología usada

1.2.1 Lo que sí se considera fiable

La verificación backend fiable se obtuvo ejecutando las clases de test del repositorio dentro de `odoo shell` con `unittest`, en vez de confiar únicamente en `--test-enable + --test-tags`, porque en este entorno se observaron ejecuciones nativas de Odoo que daban:

```
0 failed, 0 error(s) of 0 tests
```

Es decir: aparentaban terminar bien, pero en realidad **no estaban ejecutando la suite esperada**.

1.2.2 Lo que no se considera concluyente

Los tests frontend HOOT no quedan validados de forma concluyente aquí por dos motivos:

1. **odoo shell no levanta el servidor HTTP** requerido por `HttpCase`.
 2. El runner HOOT estándar llegó a mezclar suites ajenas de web, generando ruido upstream no atribuible al repositorio.
-

1.3 3. Inventario general por archivo

#	Archivo	Tests	Estado	Observaciones
1	pos_conventional_cash_calculator/tests/test_cashbox_calculator.py	6	✓ OK	Backend
2	pos_conventional_cash_calculator/tests/test_pos_cash_calculator_wizard.py	30	✓ OK	Backend
3	pos_conventional_cash_drawer/tests/test_pos_conventional_cash_drawer.py	3	✓ OK	Backend
4	pos_conventional_config_user_filter/tests/test_user_filter.py	12	✗ Errores	4 errores
5	pos_conventional_core/tests/test_pos_config.py	17	✗ Errores	3 errores
6	pos_conventional_core/tests/test_pos_order.py	82	✓ OK	Backend
7	pos_conventional_core/tests/	8	✓ OK	Backend

#	Archivo	Tests	Estado	Observaciones
8	test_pos_order_completeness.py pos_conventional_core/tests/ test_pos_order_line_cost_margin.py	38	✓ OK	Backend
9	pos_conventional_core/tests/ test_pos_order_price_list.py	23	✗ Fallos	6 fallos
10a	pos_conventional_core/tests/ test_product_label_section_and_note_field.js.py (Assets)	2	✓ OK	Backend
10b	pos_conventional_core/tests/ test_product_label_section_and_note_field.js.py (H00T)	1	⚠ No concluyente	Requiere HTTP/HOOT limpio
11	pos_conventional_core/tests/ test_receipt_print.py	17	✓ OK	Backend
12	pos_conventional_core/tests/ test_res_c	13	✓ OK	Backend

#	Archivo	Tests	Estado	Observaciones
13	onfig_settings.py pos_conventional_order_barcode/tests/test_order_barcode.py	23	✓ OK	1 skipped observado
14	pos_conventional_order_barcode/tests/test_order_barcode_frontend.py	1	⚠ No concluyente	HOOT contaminado / sin HTTP
15	pos_conventional_payment_wizard/tests/test_payment_flow.py	37	✗ Fallos	3 fallos
16	pos_conventional_payment_wizard/tests/test_payment_wizard.py	65	✓ OK	Backend
17	pos_conventional_picking_integration/tests/test_picking_integration.py	19	✓ OK	Backend
18	pos_conventional_receipt_custom/tests/test_receipt_custom.py	18	✗ Fallos + error	3 fallos + 1 error
19	pos_conven	3	✓ OK	Backend

#	Archivo	Tests	Estado	Observaciones
20	tional_returns/ tests/ test_pos_conventional_returns.py pos_conventional_sale_integration/ tests/ test_sale_integration.py	12	✓ OK	Backend
21a	pos_conventional_session_management/ tests/ test_session_management.py (TestSessionManagement)	63	✗ Fallo + errores	1 fallo + 3 errores
21b	pos_conventional_session_management/ tests/ test_session_management.py (TestClosingPopupDataStructure)	17	✓ OK	Backend
22	pos_conventional_users_pin/ tests/ test_users_pin.py	18	✓ OK	Backend

1.4 4. Detalle por archivo de test

1.5 4.1

`pos_conventional_cash_calculator/tests/test_cashbox_calculator.py`

Qué prueba

El mixin de cálculo de caja:

- suma de billetes y monedas,
- totales con distintas denominaciones,
- combinaciones de importes.

Número de tests: 6 Estado:  OK (6/6)

1.6 4.2

`pos_conventional_cash_calculator/tests/test_pos_cash_calculator_wizard.py`

Qué prueba

El wizard de calculadora de caja:

- cómputo del total,
- incrementos y decrementos por denominación,
- protección para no bajar de cero,
- acciones de volver / cancelar / confirmar,
- integración con el wizard padre de cierre.

Número de tests: 30 Estado:  OK (30/30)

1.7 4.3

`pos_conventional_cash_drawer/tests/test_pos_conventional_cash_drawer.py`

Qué prueba

Apertura de cajón desde POS convencional:

- action client devuelta,

- error si falta `bridge_url`,
- registro de assets y vistas.

Número de tests: 3 **Estado:**  **OK (3/3)**

1.8 4.4 `pos_conventional_config_user_filter/tests/test_user_filter.py`

Qué prueba

Filtrado de cajas POS permitidas por usuario:

- `allowed_pos_config_ids`,
- asignación y borrado de configuraciones,
- visibilidad según rol POS user / POS manager,
- helper de acceso.

Número de tests: 12 **Estado:**  **Con errores (4 errores)**

Errores detectados

- `test_09_pos_user_only_sees_assigned_configs`
- `test_10_pos_user_with_no_allowed_configs_sees_none`
- `test_11_pos_manager_sees_all_configs`
- `test_12_can_access_pos_config_helper_matches_security_rules`

Causa visible

El helper de creación de usuarios usa un campo inválido en `res.users`:

`ValueError: Invalid field 'groups_id' in 'res.users'`

Interpretación probable

Huele a typo o API antigua: probablemente debería usarse `groups_ids`.

1.9 4.5 `pos_conventional_core/tests/test_pos_config.py`

Qué prueba

`pos.config` en modo convencional / no táctil:

- `pos_non_touch`,
- partner por defecto,

- creación y recuperación de sesión,
- open_ui,
- redirección a pedidos,
- popup de apertura.

Número de tests: 17 **Estado:** ❌ **Con errores (3 errores)**

Errores detectados

- test_08_open_ui_normal_mode_calls_super
- test_12_open_ui_touch_config_uses_standard_odoo
- test_16_open_ui_opening_control_action_false_falls_to_super

Causa visible

El `super().open_ui()` cae en la protección nativa de Odoo:

UserError: You do not have permission to open a POS session...

Interpretación

Los tests esperan el flujo estándar de Odoo, pero en este entorno el usuario de test no dispone de permisos suficientes para ese branch concreto.

1.10 4.6 pos_conventional_core/tests/test_pos_order.py

Qué prueba

El núcleo de `pos.order` convencional:

- ribbon de pagos,
- receipt data,
- defaults de sesión y partner,
- creación de pedidos,
- totales,
- unlink con sudo y multicompañía,
- pago y facturación,
- cancelación y eliminación,
- barcode acumulado,
- selector de productos POS.

Número de tests: 82 **Estado:** ✅ **OK (82/82)**

1.11 4.7 `pos_conventional_core/tests/test_pos_order_completeness.py`

Qué prueba

La constraint de completitud:

- partner obligatorio,
- pricelist obligatoria,
- al menos una línea,
- bypass con `skip_completeness_check`.

Número de tests: 8 Estado:  OK (8/8)

1.12 4.8

`pos_conventional_core/tests/test_pos_order_line_cost_margin.py`

Qué prueba

Coste y margen en líneas POS:

- `total_cost`,
- `margin`,
- `margin_percent`,
- `recomputes`,
- `barcode`,
- sincronización fiscal e impuestos.

Número de tests: 38 Estado:  OK (38/38)

1.13 4.9 `pos_conventional_core/tests/test_pos_order_pricelist.py`

Qué prueba

Recalculo por tarifa:

- precio y descuento por pricelist,
- cambio de tarifa al cambiar partner,
- recálculo de líneas,

- actualización de total,
- mezcla de onchange nativo + propio.

Número de tests: 23 **Estado:** ❌ **Con fallos (6 fallos)**

Fallos detectados

- test_05_partner_with_different_pricelist_updates_order_pricelist
- test_07_clear_partner_reverts_to_session_pricelist
- test_09_partner_change_triggers_line_price_recalculation
- test_10_line_discount_is_20_after_vip_partner
- test_12_order_total_updated_after_partner_change
- test_21_native_onchange_presets_pricelist_lines_still_recalculate

Causa funcional visible

Al cambiar el partner:

- no se está actualizando la tarifa del pedido,
 - no se recalculan descuento ni líneas,
 - no cambia el total como esperan los tests.
-

1.14 4.10

`pos_conventional_core/tests/test_product_label_section_and_note_field_js.py`

Este archivo contiene 2 clases.

1.14.1 4.10.a TestPosProductLabelSectionAndNoteFieldAssets

Qué prueba

Que el parche JS y su test estén incluidos en los bundles correctos:

- `web.assets_backend`,
- `web.assets_unit_tests`.

Número de tests: 2 **Estado:** ✅ **OK (2/2)**

1.14.2 4.10.b TestPosProductLabelSectionAndNoteFieldHoot

Qué prueba

El test HOOT del parche JS.

Número de tests: 1 **Estado:** ⚠️ **No ejecutable limpiamente en odoo shell**

Error visible

AttributeError: 'NoneType' object has no attribute 'server_port'

1.15 4.11 pos_conventional_core/tests/test_receipt_print.py

Qué prueba

Flujo de impresión y factura simplificada tras validar o cobrar:

- creación de `account.move`,
- estado `posted`,
- `move_id` en la acción,
- render del reporte,
- contenido HTML del ticket/factura.

Número de tests: 17 **Estado:** ✅ **OK (17/17)**

Observación

Aparecen warnings de `wkhtmltopdf` con `ConnectionRefusedError`, pero la suite termina en **OK**.

1.16 4.12 pos_conventional_core/tests/test_res_config_settings.py

Qué prueba

`res.config.settings` de POS convencional:

- reflejo de `pos_non_touch`,
- propagación al config,
- bloqueo si hay sesiones abiertas,
- casos sin config o sin cambios.

Número de tests: 13 **Estado:** ✅ **OK (13/13)**

1.17 4.13 `pos_conventional_order_barcode/tests/test_order_barcode.py`

Qué prueba

Backend de barcode:

- búsqueda por barcode o `default_code`,
- creación e incremento de línea,
- restricciones por estado,
- persistencia de valores calculados,
- acumulación de escaneos.

Número de tests: 23 **Estado:**  **OK (23/23)**

Observación

Se observó **1 skipped** en esta clase.

1.18 4.14

`pos_conventional_order_barcode/tests/test_order_barcode_frontend.py`

Qué prueba

Frontend HOOT del controlador de barcode en formulario.

Número de tests: 1 **Estado:**  **No validado limpiamente**

Situación observada

- En `odoo shell`: falla `setUpClass` por ausencia de servidor HTTP.
 - En runner HOOT global: la ejecución quedó mezclada con suites `@web/...` ajenas al repositorio.
-

1.19 4.15 `pos_conventional_payment_wizard/tests/test_payment_flow.py`

Qué prueba

Flujo completo de pago:

- tarjeta y efectivo,
- wizard,
- new order,

- print receipt,
- cambio a devolver,
- parámetros enviados al frontend.

Número de tests: 37 **Estado:**  **Con fallos (3 fallos)**

Fallos detectados

- test_33_cash_with_change_includes_cash_change_in_params
- test_34_cash_with_change_includes_currency_symbol_in_params
- test_37_cash_with_change_in_print_receipt_next_action_params

Causa visible

La acción ya no envía los parámetros esperados por los tests:

- cash_change
- cash_change_currency

Y está enviando en su lugar:

- previous_sale_change
- previous_sale_currency

Interpretación

Parece haber cambiado el contrato de params esperado por la suite.

1.20 4.16

[pos_conventional_payment_wizard/tests/test_payment_wizard.py](#)

Qué prueba

Modelos y wizard de pago:

- métodos disponibles,
- popup de pago,
- alta y borrado de payments,
- amount_due / cambio,
- quick cash mode,
- pedidos con importe cero o negativo,
- factura simplificada.

Número de tests: 65 **Estado:**  **OK (65/65)**

1.21 4.17

`pos_conventional_picking_integration/tests/test_picking_integration.py`

Qué prueba

Integración POS → venta / picking / albarán:

- `pos_enable_albaran`,
- `action_pay_account`,
- creación y confirmación de `sale.order`,
- estados vinculados,
- filtros de pagos y pedidos cerrados.

Número de tests: 19 **Estado:**  **OK (19/19)**

1.22 4.18

`pos_conventional_receipt_custom/tests/test_receipt_custom.py`

Qué prueba

Ticket / factura custom:

- URL de informe,
- envío por email,
- datos extendidos de receipt,
- detalle de impuestos,
- impresión de factura simplificada,
- títulos de factura y rectificativa,
- render del widget de pagos.

Número de tests: 18 **Estado:**  **Con 3 fallos y 1 error**

Fallos detectados

- `test_14_action_print_factura_simplificada_no_invoice_returns_none`
- `test_16_report_shows_standard_title_for_regular_invoice`
- `test_17_report_shows_rectificativa_title_for_refund_invoice`

Error detectado

- `test_18_report_renders_payments_widget_without_legacy_method`

Causas visibles

- sin factura, `action_print_factura_simplificada()` devuelve un wizard de layout en vez de None,
- en los tests 16 y 17, `order.account_move` está vacío,
- en el test 18 el helper llama mal a:

`IrActionsReport._render_qweb_html()`

Error exacto

`TypeError: IrActionsReport._render_qweb_html() missing 1 required positional argument: 'docids'`

1.23 4.19

`pos_conventional_returns/tests/test_pos_conventional_returns.py`

Qué prueba

Devoluciones convencionales:

- filtro por misma caja/config,
- creación en sesión actual,
- error si no hay líneas reembolsables.

Número de tests: 3 Estado:  OK (3/3)

1.24 4.20

`pos_conventional_sale_integration/tests/test_sale_integration.py`

Qué prueba

Integración con `sale.order` y extensión de `report_sale_details`:

- `customer_account`,
- recuentos e importes,
- apertura del `sale.order` vinculado.

Número de tests: 12 Estado:  OK (12/12)

1.25 4.21

`pos_conventional_session_management/tests/test_session_management.py`

Este archivo contiene 2 clases.

1.25.1 4.21.a TestSessionManagement

Qué prueba

Gestión de sesión, apertura/cierre y popups:

- herencia de balance,
- popup de apertura,
- wizard de cierre,
- cash in/out,
- cancelación de borradores vacíos,
- cierre desde UI,
- regresiones de UX JS/XML.

Número de tests: 63 **Estado:** ❌ **Con 1 fallo y 3 errores**

Fallo detectado

- `test_22d_opening_popup_xml_shows_blue_info_banner`

Errores detectados

- `test_27_close_session_from_ui_cancels_empty_draft_in_non_touch`
- `test_30_action_close_session_cancels_empty_draft_and_closes`
- `test_37_action_close_session_multiple_empty_drafts_all_cancelled`

Causas visibles

- El XML ya no contiene el texto esperado por el test: se esperaba Caja que vas a abrir y ahora aparece algo como Apertura Caja.
- Los tests que crean pedidos vacíos fallan por la constraint de completitud: `ValidationError: El pedido / debe tener un cliente asignado.`

1.25.2 4.21.b TestClosingPopupDataStructure

Qué prueba

Estructura backend consumida por ClosingPopup:

- claves esperadas,
- detalles de caja,

- métodos de pago,
- flujo completo de cierre,
- informe diario,
- refresco tras cash move.

Número de tests: 17 **Estado:**  **OK (17/17)**

1.26 4.22 `pos_conventional_users_pin/tests/test_users_pin.py`

Qué prueba

PIN de usuario y wizard:

- unicidad de PIN,
- cambio de usuario,
- flujo de apertura y nuevo pedido,
- field related en settings.

Número de tests: 18 **Estado:**  **OK (18/18)**

1.27 5. Fallos reales detectados

Los problemas reales actuales se concentran en los siguientes archivos:

1.27.1 5.1

`pos_conventional_config_user_filter/tests/test_user_filter.py`

- **4 errores**
- **Causa visible:** uso de `groups_id` inválido en `res.users`
- **Acción probable:** corregir helper de test o código auxiliar para usar `groups_ids`

1.27.2 5.2 `pos_conventional_core/tests/test_pos_config.py`

- **3 errores**
- **Causa visible:** `open_ui()` cae en el guard nativo de permisos POS de Odoo
- **Acción probable:** ajustar permisos de usuario de test o adaptar la expectativa de esos tests al entorno actual

1.27.3 5.3 `pos_conventional_core/tests/test_pos_order_pricelist.py`

- **6 fallos**
- **Causa visible:** cambiar partner no actualiza ni recalcula la pricelist y sus líneas como espera la suite
- **Acción probable:** revisar el onchange/override de partner y la recomputación de líneas

1.27.4 5.4

`pos_conventional_payment_wizard/tests/test_payment_flow.py`

- **3 fallos**
- **Causa visible:** el contrato de parámetros cambió de `cash_change*` a `previous_sale_*`
- **Acción probable:** decidir si el cambio era intencional; si sí, adaptar tests; si no, restaurar payload anterior

1.27.5 5.5

`pos_conventional_receipt_custom/tests/test_receipt_custom.py`

- **3 fallos + 1 error**
- **Causas visibles:**
 - o retorno distinto al esperado cuando no hay factura,
 - o `account_move` no generado o no enlazado en esos casos,
 - o llamada con firma incorrecta a `_render_qweb_html(docids, ...)`

1.27.6 5.6

`pos_conventional_session_management/tests/test_session_management.py`

- **1 fallo + 3 errores**
- **Causas visibles:**
 - o el copy del banner XML cambió,
 - o la constraint de completitud bloquea la creación de borradores vacíos usada por los tests.

1.27.7 5.7 Frontend HOOT

Archivos afectados:

- `pos_conventional_core/tests/test_product_label_section_and_note_field_js.py`
- `pos_conventional_order_barcode/tests/test_order_barcode_frontend.py`

Situación: no son ejecutables limpiamente con esta técnica y la alternativa observada quedó contaminada por suites upstream de web.

1.28 6. Conclusión

1.28.1 Lo que está bien

La mayor parte del backend del repositorio está razonablemente sana:

- calculadora de caja ✓
- cash drawer ✓
- núcleo pos . order ✓
- constraint de completitud ✓
- coste / margen ✓
- impresión base ✓
- settings ✓
- barcode backend ✓
- payment wizard base ✓
- integración con picking ✓
- devoluciones ✓
- integración con ventas ✓
- estructura de closing popup ✓
- PIN de usuarios ✓

1.28.2 Lo que está roto o desalineado ahora mismo

Los problemas se concentran en:

- filtros de usuario POS,
- open_ui / permisos,
- recalcule de pricelist por partner,
- contrato de parámetros del flujo de cambio en pagos,
- receipt_custom,
- session_management por cambio de copy/UI y por la constraint de completitud,
- tests frontend HOOT por limitación de entorno/aislamiento.

1.28.3 Lectura final

El estado del repositorio no apunta a una caída generalizada. Más bien muestra un patrón claro:

1. **La base backend principal está sana.**
 2. **Los fallos se concentran en regresiones funcionales acotadas o en tests desalineados con cambios recientes.**
 3. **La validación frontend HOOT sigue pendiente de un entorno con servidor HTTP activo y mejor aislamiento de suites.**
-

1.29 7. Próximo paso recomendado

Hay dos caminos razonables:

1.29.1 Opción A — Dejar la suite backend en verde

Arreglar, uno por uno, los bloques rotos hasta dejar el backend en verde:

1. `test_user_filter.py`
2. `test_pos_config.py`
3. `test_pos_order_pricelist.py`
4. `test_payment_flow.py`
5. `test_receipt_custom.py`
6. `test_session_management.py`

1.29.2 Opción B — Preparar tablero ejecutivo más compacto

Generar una tabla de seguimiento más breve, por ejemplo CSV/Markdown, con:

- archivo,
 - número de tests,
 - estado,
 - fallos concretos,
 - causa probable,
 - prioridad,
 - decisión propuesta: **corregir código** vs **actualizar test**.
-

1.30 8. Estado documental

Este fichero queda como fuente canónica del informe de tests. Si se corrigen suites o se vuelve a ejecutar la validación, debe actualizarse:

- la fecha,
- el resumen ejecutivo,
- el inventario por archivo,
- y la sección de fallos reales detectados.